

Hack-AI-thon Screening Assignment

AI ScrumOS — An AI-native Engineering Operating System that Autonomously Manages Software Delivery Workflows

Candidate: Ayush Sharma
GitHub: github.com/Ayushrak/AIScrumOS
Repository: <https://github.com/Ayushrak/AIScrumOS>
Tech Stack: React + Vite (Frontend) | Python FastAPI (Backend) | WebSockets
Date: 24th May 2026

1. LLM Project — AI ScrumOS

1.1 What Problem the System Solves

Traditional project management tools (Jira, Linear, Asana) are **reactive** — they require manual ticket updates, manual follow-ups, and developer overhead. In modern engineering teams, **70% of developer blockers are never explicitly raised**. Developers silently struggle with environment crashes, configuration loops, and dependency failures without logging a blocker, causing sprint delays that are only discovered at the end.

AI ScrumOS solves this by acting as an **autonomous AI Engineering Manager** that continuously ingests real-time telemetry — Git commit patterns, CI/CD pipeline statuses, Slack channel activity, meeting transcriptions, and developer work-hour distributions — to:

- **Detect silent blockers** using behavioral signal analysis (repeated build failures, config file edits, no git pushes).
- **Predict sprint delivery risks** by computing cascading delay probabilities across microservice dependency graphs.
- **Auto-coordinate resolutions** by creating Jira tickets, sending Slack notifications, and assigning domain experts.
- **Preserve engineering memory** via a RAG-powered temporal search system that indexes historical Slack chats, Jira tickets, GitHub PRs, and retrospective action items.
- **Monitor team wellness ethically** by tracking context-switching indexes, late-night commit patterns, and workload ratios.

1.2 High-Level Architecture & Key Components

AI ScrumOS is built on a **decoupled client-server architecture** with two independent services communicating via REST APIs and WebSocket event streams:

Layer	Technology	Responsibility
Frontend	React 19 + Vite 8 + Vanilla CSS	Glassmorphic cyberpunk dashboard with interactive SVG dependency graphs, real-time WebSocket event listeners, and a 6-step simulator control panel.

Backend	Python FastAPI + Uvicorn + Pydantic Settings	REST API gateway, WebSocket connection manager for broadcasting live events, state machine agent pipelines (Sprint Agent, Risk Agent, Memory Agent), and environment-based configuration loading.
Intelligence	Mock Agent State Machine + RAG Emulation	Simulates AI agent coordination loops. Computes cumulative sprint state transitions, burnout risk matrices, and keyword-matching temporal search over indexed engineering decisions.

Key Data Flow: The React client connects to the FastAPI WebSocket endpoint (`/ws/events`) on mount. When a judge clicks 'Advance Step' on the simulator, the frontend sends a POST request to `/api/simulator/step`. The backend processes the state transition through the agent pipeline, updates the in-memory database, and **broadcasts** the new state to all connected WebSocket clients simultaneously — enabling real-time, multi-client dashboard updates.

1.3 Engineering Challenge — Real-Time Fallback Architecture

The most significant engineering challenge was designing a **zero-downtime fallback system** for live hackathon presentations. During a stage demo, the Python backend server might crash, lose connection, or fail to start. If the React dashboard simply displayed an error screen, the entire presentation would be ruined.

Solution: I implemented a **dual-mode state architecture** in the React client. On mount, the app attempts to connect to the FastAPI server via a health-check fetch to `/api/sprint-data`. If the backend responds, the app enters **Backend Mode** — all state is fetched from REST APIs, and the WebSocket channel streams live event logs. If the backend is unreachable, the app transparently falls back to **Local Simulation Mode** — computing identical state transitions using a pure JavaScript state machine bundled in `mockData.js`. The UI displays a subtle status indicator showing which mode is active, but the interactive experience remains identical. This means **the demo never breaks**, regardless of server availability.

1.4 Repository & Demo

GitHub Repository: <https://github.com/Ayushrak/AIScrumOS>

The repository includes a comprehensive README.md with setup instructions, architecture diagrams (Mermaid), full directory structure documentation, and a generated cyberpunk UI preview image.

2. Architecture Design — Document Q&A; System

Below is my proposed pipeline for a production-grade Document Q&A; system over 10,000 internal documents (PDFs, Word files, and Confluence pages) with semantic retrieval and cited answers.

2.1 Chunking Strategy

I would use **Recursive Character Text Splitting** (LangChain's RecursiveCharacterTextSplitter) with a chunk size of **800–1200 tokens** and an overlap of **150–200 tokens**. This splitter respects natural document boundaries (paragraphs → sentences → words) rather than splitting mid-sentence, which preserves semantic coherence. For structured documents (Confluence pages with headers), I would additionally use **Markdown/HTML header-based splitting** to create chunks aligned to document

sections, attaching section titles as metadata for improved retrieval context.

2.2 Embedding Model Choice

OpenAI text-embedding-3-small (1536 dimensions) for the best cost-performance ratio at scale, or **text-embedding-3-large** (3072 dimensions) if accuracy is paramount. For an on-premise or open-source alternative, **BGE-large-en-v1.5** or **E5-large-v2** from HuggingFace offer competitive MTEB benchmark scores without vendor lock-in. The embedding model must be consistent between ingestion and query time.

2.3 Vector Store Choice

Pinecone for managed cloud deployment (serverless scaling, metadata filtering, and hybrid search support), or **Qdrant** for self-hosted requirements (rich filtering, payload indexing, and excellent Rust-based performance). For prototyping, **ChromaDB** works well locally. At 10,000 documents with ~100K chunks, any modern vector database handles the scale comfortably.

2.4 Retrieval Strategy

I would implement a **hybrid retrieval pipeline** combining multiple strategies:

- **Semantic Search:** Dense vector similarity (cosine) over the embedded chunks — handles paraphrased and conceptual queries.
- **BM25 Keyword Search:** Sparse retrieval for exact term matching — handles acronyms, product codes, and proper nouns that embeddings may miss.
- **Reciprocal Rank Fusion (RRF):** Merges ranked results from both retrievers into a single ordered list, combining the strengths of semantic and lexical search.
- **Metadata Filtering:** Pre-filter by document type, department, or date range to narrow the search scope before vector similarity.
- **Re-ranking:** Apply a cross-encoder re-ranker (e.g., Cohere Rerank or a ColBERT model) on the top-20 retrieved chunks to produce a final top-5 with higher precision.

2.5 Handling Partial Answers

When no single document fully answers a question, I would implement **multi-document synthesis**: retrieve the top-K chunks (K=5–8), pass all of them to the LLM with a prompt that instructs it to synthesize a comprehensive answer by combining information across sources. Each claim in the response must cite its source chunk using bracketed references (e.g., [Doc A, Section 3]). The prompt explicitly instructs the model to say 'Based on available documents, this is only a partial answer' when coverage is incomplete, rather than hallucinating.

2.6 Biggest Failure Mode & Mitigation

Biggest Failure Mode: Retrieval misses — the correct document chunks are not retrieved because the user's query is semantically distant from the document's language (vocabulary mismatch), or the answer spans multiple chunks that individually score low in isolation.

Mitigation: (1) **Query expansion** — use an LLM to rewrite the user's question into 3–4 alternative phrasings before retrieval, then union the results. (2) **Hypothetical Document Embedding (HyDE)** — generate a hypothetical answer first, embed it, and use that embedding for retrieval (often more effective than embedding the question directly). (3) **Parent-child chunking** — store small chunks for precise

retrieval but return the larger parent chunk for context, ensuring the LLM sees enough surrounding information. (4) Implement **user feedback loops** — log queries with low confidence scores for human review and continuous improvement.

3. Debug Challenge — Find and Fix the Bugs

Below is my analysis of every bug in the submitted RAG pipeline code, explaining what each issue actually causes in production, followed by the corrected version.

Bug 1: `chunk_overlap > chunk_size` (Runtime Crash)

`chunk_size=100, chunk_overlap=500` — The overlap (500) exceeds the chunk size (100). This raises a **ValueError** at runtime because it is logically impossible for the overlap region to be larger than the chunk itself. The splitter cannot create valid windows.

Fix: Set `chunk_size=1000, chunk_overlap=200`. A chunk size of 1000 characters with 200 overlap ensures sufficient semantic context per chunk while maintaining continuity between adjacent chunks.

Bug 2: Wrong Embedding Model (Silent Wrong Answers)

`OpenAIEmbeddings(model="gpt-4")` — GPT-4 is a **chat completion model**, not an embedding model. This will either throw an API error or silently return garbage vectors. Embeddings require a dedicated embedding model.

Fix: Use `OpenAIEmbeddings(model="text-embedding-3-small")` or `text-embedding-ada-002`.

Bug 3: `temperature=1.0` (Non-Deterministic, Hallucination-Prone)

`temperature=1.0` — Maximum randomness for a factual Q&A; system. This causes the LLM to generate creative, varied responses instead of grounded, deterministic answers. In a RAG pipeline this dramatically increases **hallucination risk** — the model may invent facts not present in the retrieved documents.

Fix: Set `temperature=0.0` (or at most 0.1) for factual retrieval tasks.

Bug 4: `k=50` Retrieved Documents (Context Overflow)

`search_kwargs={"k": 50}` — Retrieving 50 chunks will likely **exceed the LLM's context window**, causing a token limit error. Even if it fits, the LLM's attention degrades with excessive context (the 'lost in the middle' problem), producing worse answers. It also increases API cost and latency dramatically.

Fix: Set `k=3` to `k=5`. Use a re-ranker if precision matters.

Bug 5: `return_source_documents=False` (No Citations)

Setting `return_source_documents=False` discards the source chunks used to generate the answer. This means **users cannot verify the answer's accuracy**, there is no traceability, and the system cannot display citations — a fundamental requirement for any enterprise Q&A; system.

Fix: Set `return_source_documents=True` and display source metadata alongside answers.

Bug 6: No Error Handling (Production Crash)

`qa_chain.run(...)` has **zero error handling**. In production, API rate limits, network timeouts, malformed documents, and OpenAI outages will crash the entire application. The `.run()` method is also deprecated in favor of `.invoke()`.

Fix: Wrap in try/except, implement retries with exponential backoff, use `.invoke()`, and log errors.

Bug 7: Single File Loader (Architectural Limitation)

`TextLoader("company_docs.txt")` loads a single plaintext file. A production system with multiple document types (PDFs, Word, Confluence) needs **DirectoryLoader** with appropriate per-format parsers (PyPDFLoader, Docx2txtLoader, ConfluenceLoader). This is an architectural gap that will surface immediately when the system is deployed against real document corpora.

Fix: Use `DirectoryLoader` with glob patterns and format-specific loader mappings.

Bug 8: CharacterTextSplitter (Poor Semantic Chunking)

`CharacterTextSplitter` splits on a single character (default: `'\n\n'`). This often produces chunks that break mid-sentence or mid-paragraph, destroying semantic coherence. Under load with diverse document formats, this leads to **degraded retrieval quality** because chunks contain fragmented, incomplete thoughts.

Fix: Use `RecursiveCharacterTextSplitter` which tries multiple separators (paragraphs → sentences → words) to find the most natural split points.

Corrected Production Code

```
from langchain_community.document_loaders import DirectoryLoader, TextLoader
from langchain.text_splitter import RecursiveCharacterTextSplitter
from langchain_openai import OpenAIEmbeddings, ChatOpenAI
from langchain_community.vectorstores import Chroma
from langchain.chains import RetrievalQA
import logging

logging.basicConfig(level=logging.INFO)
logger = logging.getLogger(__name__)

# Load documents from directory
loader = DirectoryLoader('company_docs/', glob='**/*.txt', loader_cls=TextLoader)
docs = loader.load()

# Recursive splitting with proper sizes
splitter = RecursiveCharacterTextSplitter(
    chunk_size=1000,
    chunk_overlap=200,
    separators=['\n\n', '\n', '. ', ' ', '']
)
chunks = splitter.split_documents(docs)

# Correct embedding model
embeddings = OpenAIEmbeddings(model='text-embedding-3-small')
vectorstore = Chroma.from_documents(chunks, embeddings,
    persist_directory='./chroma_db')

# Deterministic LLM for factual Q&A
```

```

llm = ChatOpenAI(model='gpt-4', temperature=0.0)
qa_chain = RetrievalQA.from_chain_type(
    llm=llm,
    retriever=vectorstore.as_retriever(search_kwargs={'k': 4}),
    return_source_documents=True,
    chain_type='stuff'
)

# Query with error handling
try:
    result = qa_chain.invoke({'query': 'What is our refund policy?'})
    print(result['result'])
    for doc in result['source_documents']:
        print(f'Source: {doc.metadata}')
except Exception as e:
    logger.error(f'RAG pipeline error: {e}')
    print('Unable to process query. Please try again.')

```